PRACTICE EXERCISE 8: DATA STORAGE – SQLITE DATABASE

LEARNING OBJECTIVES

- **Understanding SQLite:**
  - Learn what SQLite is, its features, and why it's used in Android development.
  - Understand SQLite's limitations and strengths compared to other databases.
- **Database Basics:**
  - Understand fundamental database concepts such as tables, columns, rows, primary keys, and foreign keys.
  - Learn about SQL (Structured Query Language) and how to use it to perform CRUD (Create, Read, Update, Delete) operations.
- **Android Integration:**
  - Learn how to integrate SQLite into Android applications.
  - Understand the role of SQLiteOpenHelper and SQLiteDatabase classes in managing database creation, upgrades, and access.
- **Database Operations:**
  - Learn how to perform basic database operations like creating tables, inserting data, querying data, updating records, and deleting records.
  - Explore more advanced operations such as transactions, joining tables, and indexing for performance optimization.
- **Error Handling and Data Integrity:**
  - Learn about error handling strategies when working with databases, including handling exceptions and ensuring data integrity.
  - Understand concepts like database transactions and how they help maintain data consistency.
- **Security:**
  - Understand security considerations when working with SQLite databases in Android, including preventing SQL injection attacks and securing sensitive data.

- **Real-World Application:**
  - Apply your knowledge by building a real-world Android application that uses SQLite for data storage, such as a note-taking app or a simple task manager.

OVERVIEW

SQLite is a lightweight database management system integrated into Android, suitable for storing local data in mobile applications. To use SQLite in Android, you need to use the ***SQLiteOpenHelper*** class to manage the database and perform operations such as insertion, querying, updating, and deletion using SQL or methods provided by the SQLiteDatabase class. SQLite provides lightweight, fast, and serverless database management, optimizing performance and reducing costs in Android app development.

TERMINOLOGY

SQLite: A lightweight, serverless, and self-contained relational database management system.

SQLiteOpenHelper: A helper class to manage database creation and version management.

SQLiteDatabase: The primary class for managing a SQLite database and performing various operations like insert, update, delete, and query.

Schema: The structure of the database, including tables, columns, and constraints.

Table: A collection of related data stored in rows and columns.

Column: A named element within a table that stores a specific type of data.

Row: A single record or entry within a table.

Primary Key: A unique identifier for each row in a table.

Foreign Key: A column in a table that references the primary key of another table to establish a relationship between them.

CRUD Operations: Acronym for Create, Read, Update, and Delete operations, the basic operations performed on a database.

Transaction: A unit of work performed within a database, typically involving multiple operations that must either all succeed or all fail.

Index: A data structure that improves the speed of data retrieval operations on a database table at the cost of additional space and slower write operations.

SQL Injection: A security vulnerability that occurs when an attacker inserts malicious SQL code into input fields to manipulate the database.

Data Integrity: The accuracy and consistency of data stored in a database, maintained through constraints, relationships, and transactions.

ORM (Object-Relational Mapping): A programming technique used to convert data between incompatible systems by mapping object-oriented programming language data to a relational database model.

8.1. THEORETICAL SUMMARY

8.1.1. Knowledge

When working with SQLite databases in Android development, it's essential to have knowledge in several areas:

SQL (Structured Query Language): Understanding SQL is crucial as it's the language used to interact with SQLite databases. Learn about SQL syntax, data manipulation commands (SELECT, INSERT, UPDATE, DELETE), data definition commands (CREATE TABLE, ALTER TABLE, DROP TABLE), and constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL).

Android Development Basics: Familiarity with Android development fundamentals, including activities, fragments, layouts, and intents, is necessary to integrate SQLite databases into Android applications seamlessly.

Java or Kotlin Programming: Since Android apps are typically developed using Java or Kotlin, proficiency in one of these programming languages is essential. You'll need to write code to interact with SQLite databases using Java/Kotlin classes and methods.

SQLiteOpenHelper and SQLiteDatabase: Understand the usage of these classes provided by the Android framework for managing SQLite databases, including database creation, versioning, and performing CRUD operations.

Database Design Principles: Learn about database design principles such as normalization, denormalization, indexing, and optimizing queries. This knowledge helps in designing efficient and scalable database schemas.

Error Handling and Data Validation: Understand how to handle errors and validate data input to ensure data integrity and prevent security vulnerabilities such as SQL injection attacks.

8.1.2. Skill

The essential skills required for working with SQLite databases in Android development include:

- Proficiency in writing SQL queries to interact with the database.
- Strong knowledge of Android app development and utilization of platform components.
- Deep understanding of SQLiteOpenHelper and SQLiteDatabase classes for database management.
- Ability to design efficient database schemas based on database design principles.
- Skills in error handling and data validation to ensure data integrity and security.
- Management of database transactions to maintain data consistency.
- Application of security methods to protect data and users.
- Testing and debugging skills to ensure accuracy and reliability of the application.
- Optional: Familiarity with the Room Persistence Library to simplify database interactions.

8.2. BASIC PRACTICE

8.2.1. Exercise 1

Write an application that use Shared preferences to Read/Write data using SQLite.

Please follow these steps:

Step 1: Create a New Project in Android Studio

Create new project with name : LabEight. (Should choose Empty Activity, don't need to use other layout temmplates)

Step 2: UX/UI Design:

Next, update activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="15dp"
        android:text="@string/hello_world"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <EditText
        android:id="@+id/editTextTextMultiLine"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="48dp"
        android:autofillHints=""
        android:ems="10"
        android:gravity="start|top"
        android:hint="@string/your_content"
        android:inputType="textMultiLine"
        android:lines="10"
        android:maxLines="150"
        android:minHeight="48dp"
        android:textColorHint="#757575"
        app:layout_constraintStart_toStartOf="@+id/textView"
        app:layout_constraintTop_toTopOf="@+id/textView" />

    <Button
        android:id="@+id/buttonRead"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="@string/read"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/editTextTextMultiLine" />

    <Button
        android:id="@+id/buttonWrite"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:layout_marginTop="15dp"
        android:text="@string/write"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonRead" />

    <EditText
        android:id="@+id/editTextTextMultiLine2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="20dp"
        android:autofillHints=""
        android:ems="10"
        android:gravity="start|top"
        android:hint="@string/result"
        android:inputType="textMultiLine"
        android:lines="3"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Next, update strings.xml

```
<resources>
    <string name="app_name">LabEight</string>
    <string name="hello_world">Hello World! LabEight</string>
    <string name="your_content">Type your content here</string>
    <string name="read">Read</string>
    <string name="write">Write</string>
    <string name="result">result here</string>
</resources>
```

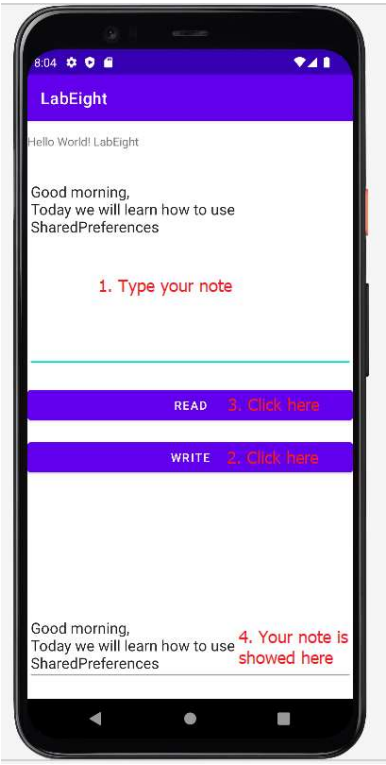
Step 3: Handling events

Next, update MainActivity.java

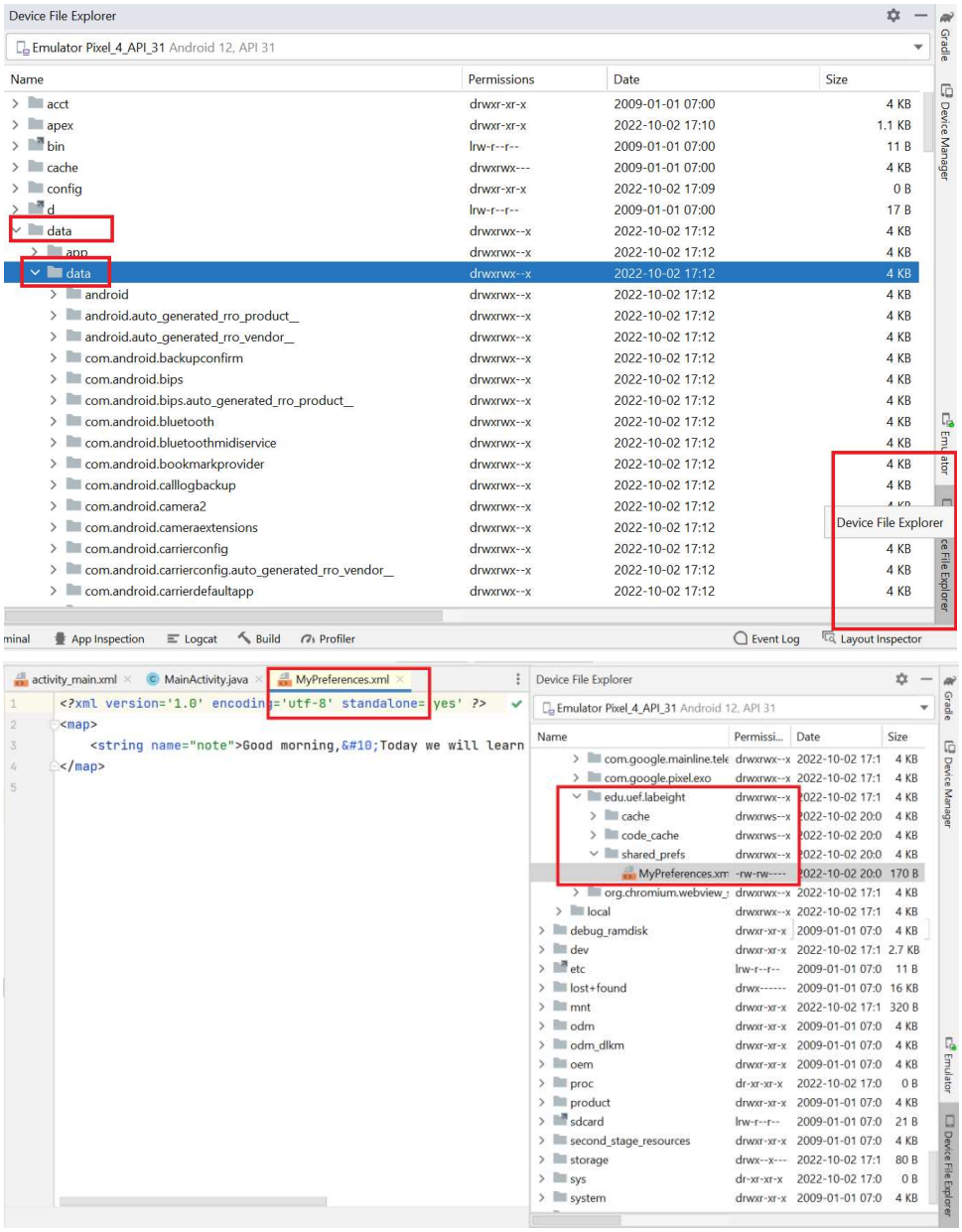
```
package edu.uef.labeight;

import androidx.appcompat.app.AppCompatActivity;

import android.content.SharedPreferences;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;

public class MainActivity extends AppCompatActivity {
    Button bt_read,bt_write;
    EditText et_input;
    EditText et_result;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        et_input=(EditText)findViewById(R.id.editTextTextMultiLine);
```

```
        et_result=(EditText)findViewById(R.id.editTextTextMultiLine2);
        bt_read=(Button)findViewById(R.id.buttonRead);
        bt_write=(Button)findViewById(R.id.buttonWrite);

        bt_write.setOnClickListener(view -> {
            String noteInput=et_input.getText().toString();

            SharedPreferences sharedPrefWrite=
getSharedPreferences("MyPreferences", MODE_PRIVATE);
            SharedPreferences.Editor editor=sharedPrefWrite.edit();
            editor.putString("note", noteInput);
            editor.apply();     //Update data don't return anything
            //editor.commit();   //Update data return:True/False
        });

        bt_read.setOnClickListener(view -> {
            SharedPreferences sharedPref=
getSharedPreferences("MyPreferences", MODE_PRIVATE);
            String noteContent=sharedPref.getString("note", "Your note");
            et_result.setText(noteContent);
        });
    }
}
```

Next, run your application



Find your stored data in android device: Open “Device File Explorer” and find your package name



Continue adding some functionalities to the application:

Write a program to create a File and write it to Internal Storage. Check File in "File Explorer". Write your Note into YourNote.txt file.

Tips:

+ Using FileOutputStream with MODE_PRIVATE
+ Using write() close() to store data and close stream.
+ Check file in "File Explorer" with your package name.

Write a program to create a File and Read/write it to External Storage (SD Card). Check File in "File Explorer".

Write your Note into YourNote.txt file if your SDCard available

Tips:

+ Create function that check your SDCard status. Using

```
Environment.getExternalStorageState()
```

+ Check permission. Register in AndroidManifest file:

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

+ Using FileOutputStream. Using read() write() close() to working with data and close stream.

+ Get file directory:

```
Environment.getExternalStorageDirectory()
```

+ Check file in "File Explorer" with your package name.

Now, We upgrade your current project with SQLite Databases

Extends SQLiteOpenHelper to create/update tables. Pass Database name and version into super() function. We need to override onCreate() and onUpgrade(). onCreate() to be called if database does not exist. Using onUpgrade() if database version changes. Access the database in read or write mode by using getReadableDatabase() and getWriteableDatabase().

Next, update AndroidMainfest.xml

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

Next, update activity_main.xml layout:

Create new button that constraint with ="@+id/buttonWrite and update strings.xml

```
<Button
    android:id="@+id/buttonSaveDB"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="15dp"
    android:text="@string/button_savedb"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/buttonWrite" />
```

Next, create new java file that extends SQLiteOpenHelper with name: **"LabDatabase"**

```
package edu.uef.labeight;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class LabDatabase extends SQLiteOpenHelper {
```

```
    //Define Database
    private static final String DB_NAME="labdb.db";
    private static final int DB_VERSION=1;
    private static final String Table_NAME="notes";
    private static final String CLM_ID="_id";
    private static final String CLM_CONTENT="content";

    public LabDatabase(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
        // TODO Auto-generated constructor stub
    }
    @Override
    public void onCreate(SQLiteDatabase sqLiteDatabase) {
        String sql="create table " + Table_NAME +
                "(" +
                CLM_ID +" integer primary key autoincrement," +
                CLM_CONTENT+" text" +
                ")";
        sqLiteDatabase.execSQL(sql);
    }
    @Override
    public void onUpgrade(SQLiteDatabase sqLiteDatabase, int oldVersion, int
newVersion) {
        sqLiteDatabase.execSQL("drop table if exists " + Table_NAME);
        onCreate(sqLiteDatabase);
    }

    public void createNote(String content)
    {
        SQLiteDatabase sqLiteDatabase=this.getWritableDatabase();
        ContentValues myData=new ContentValues();
        myData.put(CLM_CONTENT,content);
        String nullcolumnhack=null;
        sqLiteDatabase.insert(Table_NAME, nullcolumnhack, myData);
        sqLiteDatabase.close();
    }
    public Cursor getAllNotes()
    {
        SQLiteDatabase db=this.getReadableDatabase();
        Cursor c=db.rawQuery("select * from " + Table_NAME,null);
        return c;
    }
    public int countRecord()
    {
        SQLiteDatabase db=this.getReadableDatabase();
        Cursor c=db.rawQuery("select count(*) from " + Table_NAME,null);
        c.moveToFirst();
        int x;
        do
        {
            x=Integer.parseInt(c.getString(0));
        }while(c.moveToNext());
        return x;
    }
}
```

Next, update MainActivity

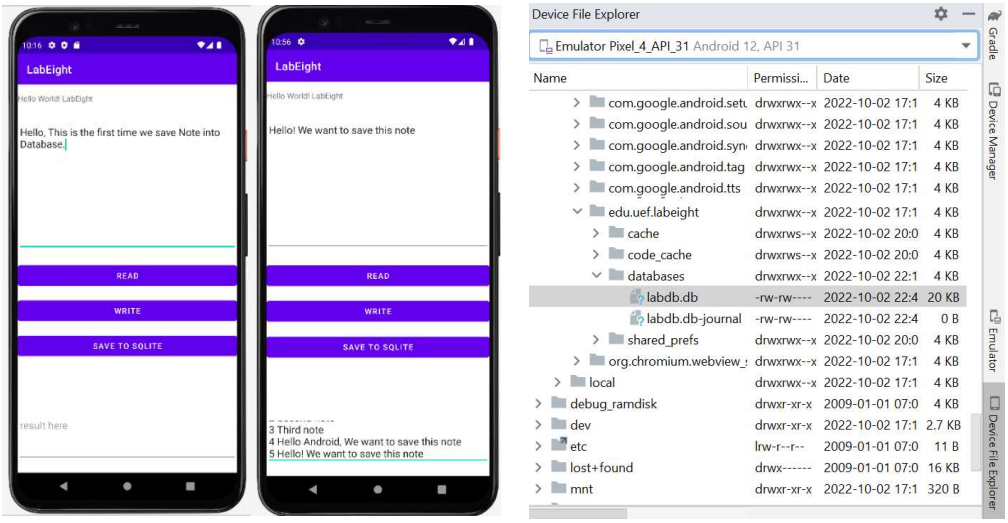
```
Declare: Button bt_saveDB;
```

```
Mapping: bt_saveDB=(Button)findViewById(R.id.buttonSaveDB);
```

Hadle click event: When click SaveDB button, create new Database if the first time we launch app or don't have any content before

```
bt_saveDB.setOnClickListener(view -> {
    String noteInput=et_input.getText().toString();
    LabDatabase db=new LabDatabase(this);
    if(db.countRecord()==0)
    {
        db.createNote("First note");
        db.createNote("Second note");
        db.createNote("Third note");
    }
    else{
        db.createNote(noteInput); // save your note
    }
    // After save note into DB, show them in Result
    Cursor c=db.getAllNotes();
    c.moveToFirst();
    String noteContent="";
    do
    {
        noteContent+=c.getString(0)+ " ";
        noteContent+=c.getString(1)+"\n";
    }while(c.moveToNext());
    et_result.setText(noteContent);
    db.close();
    c.close();
});
```

Next, run your application: Type your note and Click **"SAVE TO SQLITE"** button:



You continue to complete the project and then upload the source code to GitHub

8.2.2. Exercise 2

Create a To-Do List App: Develop a simple to-do list application where users can add, edit, delete, and mark tasks as completed. Use SQLite to store the tasks data. Below is a simplified example of how you can implement the "To-Do List App" features in Android using Java:

Please follow these steps:

Step 1: Create a New Project in Android Studio

Create new project with name : **labEightAdv.** (Should choose Empty Activity, don't need to use other layout temmplates)

Step 2: UX/UI Design

The activity_main.xml file contains an EditText for entering task titles, a Button to add tasks, and a ListView to display the list of tasks: ***activity_main.xml (Layout for MainActivity)***

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <EditText
        android:id="@+id/edit_text_task"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter task title"
        android:imeOptions="actionDone"
        android:singleLine="true" />

    <Button
        android:id="@+id/button_add_task"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/edit_text_task"
        android:layout_marginTop="8dp"
        android:text="Add Task" />

    <ListView
        android:id="@+id/list_view_tasks"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@id/button_add_task"
        android:layout_marginTop="16dp" />

</RelativeLayout>
```

The list_item_task.xml file defines the layout for each item in the ListView, containing a TextView to display the task title and a CheckBox to mark the task as completed ***list_item_task.xml (Layout for ListView item):***

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="8dp">
    <TextView
        android:id="@+id/text_view_task_title"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="18sp"
        android:textStyle="bold" />
    <CheckBox
        android:id="@+id/checkbox_task_completed"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Done" />
</LinearLayout>
```

Step 3: Handling events

Task Model Class

```
public class Task {
    private int id;
    private String title;
    private boolean completed;
    // Constructors, getters, and setters
}
```

Database Helper Class

```
public class DatabaseHelper extends SQLiteOpenHelper {
         private static final String DATABASE_NAME = "tasks.db";
         private static final int DATABASE_VERSION = 1;
         private static final String TABLE_TASKS = "tasks";
         private static final String COLUMN_ID = "id";
         private static final String COLUMN_TITLE = "title";
         private static final String COLUMN_COMPLETED = "completed";

         public DatabaseHelper(Context context) {
             super(context, DATABASE_NAME, null, DATABASE_VERSION);
         }
         @Override
         public void onCreate(SQLiteDatabase db) {
             String CREATE_TASKS_TABLE = "CREATE TABLE " + TABLE_TASKS +
                     "(" +
```

```
                    COLUMN_ID + " INTEGER PRIMARY KEY," +
                    COLUMN_TITLE + " TEXT," +
                    COLUMN_COMPLETED + " INTEGER DEFAULT 0" +
                    ")";
            db.execSQL(CREATE_TASKS_TABLE);
        }
        @Override
        public void onUpgrade(SQLiteDatabase db, int oldVersion, int
newVersion) {
            db.execSQL("DROP TABLE IF EXISTS " + TABLE_TASKS);
            onCreate(db);
        }
        // CRUD operations
        // Add new task
        public void addTask(Task task) {
            SQLiteDatabase db = this.getWritableDatabase();
            ContentValues values = new ContentValues();
            values.put(COLUMN_TITLE, task.getTitle());
            db.insert(TABLE_TASKS, null, values);
            db.close();
        }
        // Get single task
        public Task getTask(int id) {
            SQLiteDatabase db = this.getReadableDatabase();
            Cursor cursor = db.query(TABLE_TASKS, new String[]{COLUMN_ID,
COLUMN_TITLE, COLUMN_COMPLETED}, COLUMN_ID + "=?",
                    new String[]{String.valueOf(id)}, null, null, null, null);
            if (cursor != null)
                cursor.moveToFirst();
            Task task = new Task();
            task.setId(Integer.parseInt(cursor.getString(0)));
            task.setTitle(cursor.getString(1));
            task.setCompleted(cursor.getInt(2) == 1);
            cursor.close();
            return task;
        }
        // Get all tasks
        public List<Task> getAllTasks() {
            List<Task> taskList = new ArrayList<>();
            String selectQuery = "SELECT * FROM " + TABLE_TASKS;
            SQLiteDatabase db = this.getWritableDatabase();
            Cursor cursor = db.rawQuery(selectQuery, null);
            if (cursor.moveToFirst()) {
                do {
                    Task task = new Task();
                    task.setId(Integer.parseInt(cursor.getString(0)));
```

```
                task.setTitle(cursor.getString(1));
                task.setCompleted(cursor.getInt(2) == 1);
                taskList.add(task);
            } while (cursor.moveToNext());
        }
        cursor.close();
        return taskList;
    }
    // Update task
    public int updateTask(Task task) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put(COLUMN_TITLE, task.getTitle());
        values.put(COLUMN_COMPLETED, task.isCompleted() ? 1 : 0);
        return db.update(TABLE_TASKS, values, COLUMN_ID + " = ?",
                new String[]{String.valueOf(task.getId())});
    }
    // Delete task
    public void deleteTask(Task task) {
        SQLiteDatabase db = this.getWritableDatabase();
        db.delete(TABLE_TASKS, COLUMN_ID + " = ?",
                new String[]{String.valueOf(task.getId())});
        db.close();
    }
}
```

8.3. APPLICATION EXERCISES

Here are a few more applications you can consider building to practice working with SQLite databases in Android

1. **Expense Tracker**

Create an expense tracking app that allows users to log their expenses, categorize them, set budgets, and view spending trends over time.

2. **Recipe Manager**

Develop a recipe management app where users can store and organize their favorite recipes, add notes, rate recipes, and search for recipes based on ingredients or categories.

3. **Fitness Tracker**

Build a fitness tracking app that lets users log their workouts, track progress, set fitness goals, and view statistics such as calories burned, distance covered, and duration of workouts.

4. Inventory Management

Create an inventory management app for tracking items in stock, managing product details, setting reorder levels, and generating reports on inventory levels and sales.

REFERENCES

[1]. Android Developers. Data Storage. Retrieved from https://developer.android.com/guide/topics/data

[2]. Android Developers. SharedPreferences.Editor. Retrieved from https://developer.android.com/reference/android/content/SharedPreferences.Editor

[3]. Android Developers. SQLite Training. Retrieved from https://developer.android.com/training/data-storage/sqlite

[4]. Android Developers. SQLite Package Summary. Retrieved from https://developer.android.com/reference/android/database/sqlite/package-summary